

Programming Language Design

Module 9: Typeclasses

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Last module

We learned about records.

Finally, something that looks like a struct.

There were some weird wrinkles about field names though [can anyone recall?].

This module

We're going to learn about typeclasses

This is a strange topic, because you already know a lot about them, but don't realize it.

However, you will hear the word "class" and think that it involves classes, and it doesn't.

So what you think you know, you don't know, but you actually know a lot about this subject that you don't know that you know.

What is a class?

Let's talk about classes.

The regular kind. The ones that Java and C++ have.

[What is a class?]

Some valid answers:

1. A set of methods that are bundled with data that only they are allowed to modify
2. A "blueprint" for constructing instances that have certain fields and methods.
3. A way of separating public data or methods from private data or methods
4. A way of grouping data fields together (basically a struct with functions)

Why haven't we seen these in Haskell?

Classes are actually uncommon in functional languages.

They exist, but usually are only used when interacting with non-functional code.

In functional code, we don't really benefit from them.

Why?

The 3 "pillars" of OO

In several early OO-textbooks, there were three key ideas that often defined OO programming:

1. Encapsulation
2. Inheritance (alternatively, "Abstraction")
3. Polymorphism

[What do these mean?]

People started calling these "the three pillars of OO" for some reason. In my opinion, only encapsulation and polymorphism seem necessary for a language to "feel" OO. "Abstraction" is a vague term, and as a concept is common to all programming languages and paradigms.

The 3 "pillars" of OO (2)

Classes end up being a tool that satisfies all three features:

1. Classes serve as "encapsulation boundaries". They are allowed to declare that certain data is off-limits to everyone but them.
2. Classes are the things which perform inheritance. The object of inheritance was originally envisioned (by early OO language designers) as a way to reuse code. All the fields and methods of the parent class are now usable on/by the child class. (ignoring C++'s private/protected inheritance which is more akin to composition)
3. Classes can have "overridable" (aka "virtual") methods, that do different things when called on instance of inheriting classes. This enables a form of polymorphism.

With me so far?

One tool for 3 features?

Is that really good?

On the one hand, it's kind of cool that one language tool can satisfy so many requirements and enable so many abilities.

On the other hand, it kind of makes classes feel "overloaded". They do all these different things and it might be the case that we could fulfill the other requirements better if we had specific language tools for each one. (I have an example of this)

Let's start with encapsulation. What is it for? Do we still need it?

Encapsulation

Encapsulation means keeping data hidden away with clear boundaries on how and when it can be accessed. Imagine the data in a "capsule".

Why do we want to hide data away? Why do we want to stop it from being accessed?
Why do we want to stop it from being modified?

[Thoughts]

Two reasons

There are two main benefits of encapsulation:

1. To reduce coupling. I.e., external code shouldn't need to know the type or encoding of internal fields, so you can freely change them without breaking anything.
2. To reduce the danger of shared, mutable data being changed unexpectedly and violating invariants.

Coupling

The coupling issue is still something we have to worry about, even though we don't have OO classes in Haskell.

For example, I can write pi as a constant:

```
pi :: Float  
pi = 3.1415926
```

And I can write trig functions that might be used with it:

```
sin :: Float -> Float  
sin ... = ...  
  
main = print $ sin $ pi / 4.0
```

Coupling (2)

But a 32-bit Float is not really enough room to store more than 6-9 significant figures. Maybe we want a more accurate approximation of pi.

If we replace `pi :: Float` with `pi :: Double`, we create type errors. We can no longer pass `pi / 4.0` to the `sin` function, because that function takes a `Float`, not a `Double`.

The `sin $ pi / 4.0` expression and the `pi` constant are *coupled*, meaning that changing one of them can require the other to change.

Coupling (3)

Pi is such a fundamental constant, that lots of things are coupled with it.

This isn't really a problem: it's a constant, so it doesn't really change. We can just give enough digits for most problems and if someone actually needs more they can [compute the extra digits they need](#) using a function, forcing a different kind of type error.

But what about an internal value that no one else needs?

Coupling (4)

Consider making a C++ class to store data for a video game character:

```
class Character {  
private:  
    int hp; // "hit points". how much damage can be taken.  
    int mp; // "magic points". how many spells can be casted.  
};
```

If we later want to change `hp` and `mp` to be floats to allow fractional values, we can.

We made these fields private. No one else can use them.

We might still need getters so we can, e.g., draw a health bar. This isn't perfect. We can't remove all forms of coupling, but encapsulation lets us reduce it.

Does Haskell have coupling?

Haskell still has this issue. Changing code also forces changes to coupled code.

However, there is a feature we talked about briefly: [modules](#).

This feature allows function definitions to be hidden inside a module. Modules end up acting kind of like classes, except only containing static methods/fields.

Therefore, hidden definitions can be changed, and external code (outside the module) won't break as long as the exported module definitions don't change.

But what about the other reason for encapsulation? Shared mutable state?

Shared mutable state

Let's break down this term: "shared mutable state"

- State: data that determines what the program will do.
- Mutable: changeable. I.e., not constant.
- Shared: multiple functions/procedures/modules have access to it.

This kind of data is very dangerous. Consider a variable that stores whether the missile cruiser is ready to launch the missiles. It would be very problematic if that variable suddenly changed because one procedure thought it should be ready but another was currently running (e.g., missiles ignited before the missile bays opened).

Shared mutable state

Huge numbers of bugs are caused by unexpected state changes.

Some languages delicately tease out the different kinds of state. For example, Rust makes it easy to have shared immutable state, and non-shared mutable state, but requires safeguards for shared mutable state.

Haskell takes a different approach: there is no mutability.

So shared mutable state is impossible, because Haskell does not allow anything to change. Problem solved.

The result

Therefore, we don't really need the same encapsulation techniques, like classes.

If all we want to do is hide data to reduce coupling, we can use a module, which is much simpler than a class.

We don't need an encapsulation mechanism beyond that because if nothing can change anywhere, there is no need to have a special mechanism for change.

As a result, the language is just simpler.

Questions?

Something is missing

There is, however, an important facility that we haven't talked about. Something where, if Haskell didn't have it, it would be notably less ergonomic than even Java.

I'm talking about subtype polymorphism.

Note the term *subtype* here. We've been seeing a lot of polymorphism in Haskell, but not this specific kind of it.

What even is polymorphism?

Yeah, title, good question.

[What is it?]

The roots of the term "polymorphism"

If you're familiar with Greek etymologies (or if you play a lot of RPGs) you probably know that it means "many forms (ism)".

But many forms of what?

Many forms of code. Of function definitions.

Polymorphism means that a function or operator will be different depending on the types of the underlying values.

This is a vague definition, and there are actually many different kinds of polymorphism, so let's take a look at some more concrete examples.

Parametric polymorphism

We've been seeing polymorphism a lot. It's how we can write function types like:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
```

This is a kind of polymorphism: *parametric polymorphism*

The idea here is that the composition operator `.` will generate a different kind of function depending on the types of the functions it is given as arguments.

`a` , `b` , and `c` are *type parameters*. Basically variables that are filled in with types.

Why do this?

Normally, when you search for examples of polymorphism in computer science, you end up with descriptions of how it enables code that is more *flexible*. This is kind of misleading in my opinion.

For example, in this Java code, the `+` operator is polymorphic:

```
var i = 7 + 3;  
var s = "hello " + "world";
```

It's true that this is a kind of polymorphism. Specifically, *ad-hoc* polymorphism. The operator has two different meanings: addition for number types and concatenation for strings. It only has those meanings and they are completely different. It's polymorphic, but we can't really extend it.

Why do this? (2)

Back to *parametric* polymorphism, why do this?

```
(.) :: (b -> c) -> (a -> b) -> a -> c
```

If we wanted flexibility, why wouldn't the language just do this?

```
(.) :: (Anything -> Anything) -> (Anything -> Anything) -> Anything -> Anything
```

The reason: this kind of polymorphism makes the code *less* flexible. We specifically want to exclude certain kinds of types.

We want the return type of the first function to *have* to be the same as the input type of the second function. We don't want the user to have a choice, because they might compose two incompatible functions.

So what is polymorphism really?

For this reason, I don't like to define polymorphism in terms of what a function *does*.

Instead, I prefer to define it in terms of its *type*.

Therefore, my definition of polymorphism is this:

When a function changes its concrete type depending on the types of its operands.

By concrete type, I mean the most specific type. For example, in `"hello " + "world"` in Java, `+` becomes a function from a pair of strings to a string.

This definition covers all the different kinds of polymorphism we've seen.

Questions

Different kinds of polymorphism

Now let's revisit the different kinds of polymorphism and introduce the one we care about for this lecture:

1. Ad-hoc polymorphism

This is when an operator or function changes its type depending on the type of its arguments, but in a way that is specific to that exact combination of types.

For example, `1 + 2.5` in C is interpreted as a `double` because of the specific type coercion rules in C. The `1` ends up getting converted to `1.0`.

Other languages have different rules. For example, in Rust the operands all have to have the same type. You can't add a float to an integer without converting one of the arguments.

1. Ad-hoc polymorphism (2)

This kind of polymorphism doesn't have to be hardcoded.

C++ allows operators to be overloaded in an ad-hoc way. You can say "whenever we add a Foo to a Bar, use this definition of `+` instead."

Example:

```
Foo operator + (Bar a, Baz b) {  
    ...  
}
```

This says "whenever we add a `Bar` to a `Baz`, call this function and the result it returns will be a `Foo`."

This seems incredible flexible, but the types have to be known at compile time. Otherwise the compiler will select the wrong overload.

2. Parametric polymorphism

This is the kind of polymorphism where types can be parameters:

```
someParametriclyPolymorphicFunction :: a -> b -> c -> d
```

We've talked about this, but please don't think it only happens in Haskell.

In Java and C++, type parameters appear in angle brackets:

```
var l = new ArrayList<String>();  
l.add("hello");
```

Parametric types are called *generics* and Java and *templates* in C++.

Here, the `ArrayList` class uses parametric polymorphism. It's not just an `ArrayList`, it's an `ArrayList` of `String`s.

2. Parametric polymorphism

This ability makes it so that the `add` method can only take a particular type.

In this case, it takes `String`s, because we defined our `ArrayList` to be of that type.

However, if we had made an `ArrayList<Integer>`, then the `add` method would take an `Integer` instead.

We use parametric polymorphism to allow type rules to be connected between different variables. Here, we're saying the input type of the `add` function has to be the same as the type of data stored in the array list.

Before parametric polymorphism was added to Java, `ArrayLists` could store anything, and the `add` method took only an `Object`. This meant that you couldn't rely on the array only having a particular type in it, and you had to cast whenever you got something out of the array, which increased type errors.

3. Subtype polymorphism

Now we get to the primary subject of this module.

Subtype polymorphism is when a function can be called on a bunch of different types, but it does something different for each one.

This is how interfaces work in Java (or typescript for my CS 220 students). Let's remind ourselves about interfaces first, then see how there's a similar concept in Haskell.

3. Subtype polymorphism (Interfaces)

```
interface Barkable {  
    void bark();  
}
```

What does this mean?

`Barkable` is an interface. You cannot *create* a borkable like this:

```
var something = new Barkable(); // wrong, this is an error
```

Why? Because `Barkable` just means "something that can `bark`". We have not defined what barking means, only that it is some kind of thing that we might want to do. So Java wouldn't know what would happen if we wrote `something.bark()`. We haven't defined that yet.

3. Subtype polymorphism (Interfaces 2)

In order to use the interface, we have to implement it, which in Java, is something that only a class can do:

```
class Labrador implements Barkable {  
    public void bark() {  
        System.out.println("woof!");  
    }  
}
```

Labrador is something that we can create. This is permitted:

```
var sparky = new Labrador();  
sparky.bark();
```

3. Subtype polymorphism (Interfaces 3)

Where this gets interesting is when we have multiple options:

```
class Chihuahua implements Barkable {  
    public void bark() {  
        System.out.println("yarp!");  
    }  
}
```

Now, there are two different functions (technically methods, but let's keep using the term *function*): one of them belongs to `Labrador`, and the other to `Chihuahua`.

Both do different things. They print different messages. But they have the same type, so one can be substituted for another.

Questions?

Why the big deal?

You might wonder why this would be such a big deal as to get a special name.

It's because this feature requires a lot more language support than you might think.

Consider this snippet:

```
Barkable sparky = new Labrador();  
Barkable princess = new Chihuahua();  
sparky.bark();  
princess.bark();
```

This will print:

```
woof!  
yarp!
```


Why the big deal? (2)

Here's the big question I want to ask: how does Java know what to do when we write `whatever.bark();` ?

That is, [how does it know which specific method to call]?

Not quite what you think

You might think "oh, it sees that you wrote `sparky = new Labrador()`, so it knows that `sparky` is a labrador.

That is probably true in this particular case. There is an optimization called *devirtualization* where the compiler will try to trace the flow of execution so it can know for sure which method to call.

But it can't always work that way. What about here?

```
Barkable barker;  
if (Coin.flip() == Coin.HEADS) {  
    barker = new Labrador();  
} else {  
    barker = new Chihuahua();  
}  
barker.bark(); // which one gets called?
```

Not quite what you think (2)

Don't say "oh, it's not a true random number generator, so the compiler can know what's going to happen". We usually seed the RNG based on the current time.

Imagine we hooked the RNG up to a geiger counter next to a smoke detector or something. It's *random*. How can it know which function to call?

Answer: a special language feature that operates behind the scenes. In Java this is a VTable (virtual table), a table of function pointers. So barker is pointing to a particular set of function pointers that get called for each abstract method.

This is called *dynamic dispatch*. It means *dynamically* (meaning, at runtime) *dispatching* (calling) a function call to the correct definition.

Dynamic dispatch

Dynamic dispatch is required, because Java allows things like mutation.

If a `Barkable` can be changed from a `Chihuahua` to a `Labrador`, we need the ability for the method to get routed to the right implementation.

But if things are guaranteed not to change (or be generated randomly), then most of the time, the compiler can figure out what's going to happen, and resolve to a static method.

This is the case in Haskell. The language doesn't need dynamic dispatch because nothing is dynamic. As a result, polymorphism is much simpler.

Questions?

What about Haskell

Now we get to the main point: Haskell has this feature, too, but it's not called an interface. In fact, it's a bit more powerful and flexible than an interface.

The equivalent feature in Haskell is called a *typeclass*.

HUGE IMPORTANT NOTE:

`typeclass` in Haskell is similar to `interface` in Java
it is NOT similar to `class`

A few repetitions

Typeclasses are interfaces, not classes

Typeclasses are interfaces, not classes

Typeclasses are interfaces, not classes

Haskell does not have classes. It doesn't need them. It *does* need interfaces, and they are confusingly called *typeclasses*.

Typeclasses are how Haskell implements *subtype polymorphism*. Think of it as letting you implement an interface to allow types to be used in new ways, just like in Java.

Making one

With that important background out of the way, let's finally make a typeclass in Haskell:

```
class Barkable a where  
    bark :: a -> String
```

Please ignore the fact that the keyword is `class`. It's a trick! Typeclasses are like interfaces!

This code says "an `a` can be a `Barkable` if we can call `bark` on it giving a `String`."

And just like an `interface`, a typeclass is not very useful unless it has some data to apply to. But Haskell doesn't have classes, so what implements typeclasses?

Ordinary data types

Instead, we just have ordinary data types implement type classes:

```
data Labrador = Labrador
data Chihuahua = Chihuahua

instance Barkable Labrador where
    bark _ = "woof!"

instance Barkable Chihuahua where
    bark _ = "yarp!"
```

So now, if we call the function bark, the string depends on what we pass into it:

```
bark Labrador == "woof!"
bark Chihuahua == "yarp!"
```

But that raises an immediate question...

How is that different from pattern matching?

Couldn't we just do this?

```
data Dog = Labrador | Chihuahua

bark :: Dog -> String
bark Labrador = "woof!"
bark Chihuahua = "yarp!"
```

Yes...we could do that.

But there's one thing we can't do if we do things this way.

What does subtype polymorphism let us do that basic pattern-matching/case-statements don't let us do? [Anyone?]

We can't add more

Pattern matching requires us to know, in advance, all the possible cases.

We have to have a data type with exactly two constructors:

```
data Dog = Labrador | Chihuahua
```

If we add a new constructor later, we have to modify every case statement to handle that case.

This is honestly not that big a deal, especially for software maintained by one team.

However, typeclasses allow us to add new data types to the family later, without breaking any code we've already written. This is just like interfaces.

Questions?

Typeclasses in type expressions

If we inspect the type of `bark` in GHCi, we can finally see that wide arrow notation that we've probably been seeing a lot in error messages:

```
ghci> :t bark
bark :: Barkable a => a -> String
```

This notation says "*given a `Barkable` type we'll call `a`, this function will take an instance of that type and turn it into a string*"

This is very similar to `a -> String`, but that type would accept any type for `a`. The wide arrow specifies that `a` must be a type instance (what we call an "implementor") of `Barkable`.

Let's see real examples: Show and Read

In your reading, you've already seen the functions `show` and `read`.

`show` converts things to string. For example `show 7 == "7"`

`read` does the opposite. It converts strings into other things. `read "7" :: Int == 7`
(we needed the type so it would know what to convert it to)

But what is the type of `show` and `read`?

Their type

```
ghci> :t show
show :: Show a => a -> String
ghci> :t read
read :: Read a => String -> a
```

These types imply that there are typeclasses named `Show` and `Read` that `a` must be an instance of in order for us to call these functions on it.

Typeclasses are open sets. We can create new types and add them to the typeclass. Then, we'll be able to call `show` and `read` on our new type.

Let's see an example from an older lecture...

Remember this type

```
data Character =  
    Hero { name :: String, health :: Int, mana :: Int } |  
    Monster { health :: Int, mana :: Int }
```

Suppose this is a game. It would be nice to be able to convert a character to a string to save, and read it in from one to load.

You *could* make functions for that:

```
serializeCharacter :: Character -> String  
deserializeCharacter :: String -> Character
```

However, how should `ghci` know that we want to run `serializeCharacter` when it's time to print a character? And what if someone else already wrote a function that can write things to files as long as they can be converted to strings?

Instantiating `Show`

Instead, let's use the already existing typeclass `Show`, which represents things that can be turned into strings.

By using the already existing typeclass, we make our type instantly compatible with all other functions that work on instances of `Show`.

So if someone wrote a function `saveToFile :: Show a => a -> IO ()`, we could use it even though the person who made that function had never heard of our `Character` data type.

(don't worry about `IO ()` yet. That's a monad; we'll talk about those)

Show's definition

The `Show` typeclass is defined like this:

```
class Show a where  
  show :: a -> String
```

So, to make our type an instance of it, we give an instance definition like this...

An instance of Show

```
instance Show Character where
  show (Hero {name=n, health=hp, mana=mp}) =
    "Hero {name=" ++ show n ++ ", health=" ++
      show hp ++ ", mana=" ++ show mp ++ "}"
  -- as an exercise, do show (Monster ...)
```

This makes it so that if we call `show bob` on our friend `bob` from the last lecture, we get this kind of string:

```
"Hero {name = \"Bob\", health = 100, mana = 20}"
```

Notice, because we used `show` on all the values inside `bob`, they got converted to strings for us. Using `show` on a string escapes it for us, too, so even that works.

What about `Read`?

So if `Show` is the typeclass for things that can be converted into strings, `Read` is the typeclass for things that can be constructed from strings.

See if you can implement `Read` for a simpler type. Like this one:

```
data IntPair = Pair Int Int

Instance Show IntPair where
  show (IntPair x y) == show x ++ ", " ++ show y
```

Here, I've implemented `Show` so that you can see how it gets turned into a string.

Try implementing `Read` so you can turn that string back into an `IntPair`.

What about parametric types?

That's cool, but it only works for pairs of integers.

Are we supposed to implement `Show` for every kind of pair?

No. Let's make a parametric `Pair` that works for any type:

```
data Pair a = Pair a a
```

That is, `Pair 10 20` is a `Pair Int`. We've seen this a bunch of times now, but it bears repeating that Haskell lets us name a constructor with the same name as a type, and it figures out whether an expression is a value or a type from context (i.e., whether it comes after a `::`)

What about parametric types? (2)

For `Pair 10 20`, we'd like the result to be `"10, 20"`. Ideally, we'd like it to work for any "showable" type.

So `Pair 10.0 20.0` would also work: `"10.0, 20.0"`

For this, we need "inheritance":

```
instance Show a => Show (Pair a) where
  show (Pair x y) = show x ++ ", " ++ show y
```

Here, we're saying "given some type `a` which is "showable", a pair of that `a` will also be "showable". First, show the first element of it, then insert a comma and space, and then show the second element of it.

The only requirement is that we be able to call `show` on the things inside the pair.

This isn't the same as OO inheritance

You might think "oh, inheritance, I remember that."

This is different. In OO inheritance, we pull all the data fields in from the parent class, and all its methods, too. We can then override the methods we want to override.

In Haskell, typeclasses have *no data*. They are only a list of functions. This is by design. By having only functions and no data, you avoid the so-called "deadly diamond" [we can remind ourselves of what this is if there's time].

Instead, typeclass inheritance is more like a constraint. We're saying "`Pair` is only a `Show` if the type inside of it is also a `Show` .

Because we know that `a` is a `Show` , we know that it's safe to call `show x` and `show y` , and Haskell allows the code to compile.

Questions?

Having this done automatically

Converting things into strings is such a common thing to need to do, Haskell has a standard way to do it that doesn't require you to think about what the string will look like.

Just do this:

```
data Character =  
    Hero { name :: String, health :: Int, mana :: Int } |  
    Monster { health :: Int, mana :: Int }  
    deriving Show
```

The key is the *deriving clause* at the end. That says "go ahead and generate a show instance for me, automatically"

Testing it out

Let's see if it works:

```
-- (remember that records are just ordered pairs behind the scenes)
ghci> show (Hero "Bob" 100 20)
"Hero {name = \"Bob\", health = 100, mana = 20}"
ghci> show (Monster {health = 20, mana = 20})
"Monster {health = 20, mana = 20}"
ghci> show (Pair 10 20)
"Pair 10 20"
```

Haskell was automatically able to generate a `Show` implementation for us by using simple rules.

What are the rules? For ordinary data types, print the name of the constructor, then call `show` on each of the fields. For records, do that same thing, but include the braces and field names.

"deriving"

This convenient feature can be used on more than just `Show`.

For example, we can also do deriving read:

```
data Pair a = Pair a a deriving (Show, Read)
-- we can put it on one line. Useful in GHCi.
```

```
ghci> show $ Pair 10 20
"Pair 10 20"
ghci> read "Pair 10 20" :: Pair Int -- need to say what it's going to become
Pair 10 20
```

Show and Read rules

`read` inverses `show`.

That is, we expect `read . show == id`. That is, calling `show` on some showable data, and then calling `read` on the result should be the same as doing nothing (the identity function `id`)

Haskell doesn't check this anywhere. There are some programming languages where making something an instance of a typeclass also requires you to provide a proof that it follows the rule (e.g., Coq, Lean, Idris), but Haskell isn't quite that sophisticated.

Therefore, it's on you to make sure your typeclass instances make sense.

Other useful typeclasses we can "derive"

`Eq` is the typeclass for things that support equality.

If you implement this typeclass for a type, it supports `==`.

We could do it ourselves:

```
instance Eq a => Eq (Pair a) where
  (Pair a b) == (Pair x y) = a == x && b == y
```

Here we're saying "A `Pair` of two values `a` and `b` is equal to a another `Pair` of `x` and `y` if `a` equals `x` and `b` equals `y`."

Deriving `Eq`

This situation (where we want equality to mean "all the fields are equal") is so common, Haskell lets us just derive it:

```
data Pair a = Pair a a
    deriving (Show, Read, Eq)
```

Now we can compare pairs:

```
let a = Pair 10 20
    b = Pair 10 20
in  print $ a == b -- prints "True"
```

Deriving Ord

What about inequalities? `Ord` is the typeclass for things that can be ordered (with `>`, `>=`, `<`, and `<=`)

When we `derive` it, Haskell assumes we want to first compare by the first field, and if they're equal, to then compare by the second field as a tiebreaker, and so on.

So if we `deriving (Eq, Ord)` on `Pair`, we get

```
ghci> (Pair 10 20) < (Pair 11 20)
True
ghci> (Pair 10 19) < (Pair 10 20)
True
```

Important note: `Ord` inherits from `Eq`. That is, a type cannot be `Ord` without also being `Eq`. You will get an error when `deriving Ord` without also listing `Eq`.

Deriving `Ord` (2)

What if your type doesn't have numbers? It still works:

```
data Rgb = Red | Green | Blue
         deriving (Show, Read, Eq, Ord)
```

The order goes based on the order of constructors.

Examples:

```
print $ Red < Green -- True
print $ Red < Blue  -- True
print $ Green < Blue -- True
print $ Green < Red  -- False
print $ Blue < Red   -- False
```


Deriving Enum

Lastly, it's useful to have classes that can be converted to and from integers.

This typeclass has the `toEnum` and `fromEnum` functions, which provide conversions to, and from, an `Int`.

`Bool` is an instance of this typeclass: we can convert `0` and `1` into `False` and `True`

The `Enum` typeclass also requires 2 more functions: `succ` and `pred` for the successor and predecessor. This is equivalent to incrementing or decrementing the value.

Normally `succ` crashes if you call it on the last constructor and `pred` likewise on the first (i.e., don't write `succ Blue` or `pred Red`). However, you could implement your own to make it wrap instead, for modular arithmetic.

Practice: Implement `Enum` for `Rgb` without `deriving` but make `succ` and `pred` wrap.

More useful typeclasses to be familiar with

- `Num`, the class of types that support numeric operations. This is the default assumption for any arithmetic without types. `f x = x + 2 :: Num a => a -> a`
- `Integral`, the class of types that are "integer-like" and can be converted to `Integer`
- `Floating`, the class of types that are "float-like". You can use this to write code that works with both `Float` and `Double` without needing to assume one or the other.
- `Foldable`, the class of data structures that can be folded with `foldl` and `foldr`.

These are all useful, but they don't have `derive` recipes. You can't magically interpret a random data type as a floating point number with `deriving Floating`.

Questions?

Let's make a custom number

We're used to arithmetic on fix-sized integers. We know, e.g., that if we increment an integer too many times, it wraps around to the most negative integer (overflow).

We also know that if we subtract too much, most integers will wrap around to a big number. This is called negative overflow.

(this is not underflow--underflow is something that happens to floating point numbers that round toward zero. people get this term wrong so frequently that its meaning might actually change)

(in C, signed overflow is technically undefined, but this is often what ends up happening)

Let's make a custom number (2)

However, sometimes we don't want the number to wrap around.

This situation is very frequent when doing digital signal processing.

For example, if I add two sound waves together to mix them, I don't want the wavefunctions to wrap around. That would make it sound unintelligible.

Also, if I'm doing additive blending of light values (common in 3D graphics), I don't want adding two bright lights together to somehow result in a dim light.

Let's use our knowledge of typeclasses to create a new kind of saturating integer.

The `Num` typeclass

First, what is required for something to be a `Num` ?

Here is the typeclass definition ([source](#)):

```
class Num a where
  (+) :: a -> a -> a
  (-) :: a -> a -> a -- not required: default is a - b = a + negate b
  (*) :: a -> a -> a
  negate :: a -> a -- not required: default is negate x = 0 - x
  abs :: a -> a -- absolute value
  signum :: a -> a -- get the sign: signum -7 == -1
  fromInteger :: Integer -> a -- might overflow
```

Notice, `-` and `negate` are both marked as not required. In fact *one* of them is required, but then the other can be defined in terms of the one we provided.

Math enjoyers: why do you think division is not defined in the type class?

Default implementations

Normally, typeclasses list functions that need to be defined later. Like `+` here is for the person who implements the typeclass to figure out.

Adding integers is easy, but adding ratios requires making their denominators the same before doing the addition.

We are supposed to fill in a different implementation for `+` for ratios (and this has been done for us for the `Ratio` data type)

However, typeclasses can also come with *default implementations*. Which are functions that are already provided and that get copied over every time you define a new instance.

Default implementations (2)

In this case, `-` and `negate` have default implementations that are defined in terms of each other.

This means we only need to define one, and we get the other for free.

To define a default implementation, just add some function definitions after the types. [Here's an example from `Num`](#) for `-` and `negate`.

Of course, nothing stops us from providing *both* functions. We might have optimized code for both of them, which could be faster than defining one in terms of the other.

The gameplan

Our ultimate goal is to make a new type that is an instance of the `Num` typeclass

This will make it so that we can use standard operators like `+` with our type, instead of having to define new operators that only work with saturating integers.

The plan is as follows:

1. Define a new datatype to store our integer
2. Make it an instance of the `Num` datatype by providing the minimum required set of functions.

Making a new datatype

Here is a simple one:

```
newtype SatInt = SatInt Int
    deriving (Show, Eq, Ord)
```

We're using `newtype` because we only need one constructor and one data field, and using `data` would introduce overhead for the "tag" field that would be unnecessary.

We're deriving `Show` so we can easily print our new type, and `Eq` and `Ord` for comparisons.

"Saturated" ints are just like regular `Int` s, but they won't wrap around their bounds. So, what are those bounds?

Getting the bounds

Here is how we get the maximum and minimum values for the int inside the `SatInt` :

```
intMax :: Int
intMax = maxBound

intMin :: Int
intMin = minBound
```

What's going on here? Let's look at those functions:

```
minBound :: Bounded a => a
maxBound :: Bounded a => a
```

`Bounded` is a typeclass. Any data type that is bounded must implement these two constant functions: `minBound` and `maxBound` .

They aren't constants

Those are functions, not constants.

They dispatch based on the *return type*, not based on their input (which they lack)

That is, if I do something like this:

```
import Data.Int
x :: Int8
x = minBound
```

x will be `-128`. Haskell looks at the type of the scope that we call `minBound` from, sees that it's an `Int8`, and calls the `minBound` function from the `Int8` instance of `Bounded`.

With me so far?

Saturating

In order to make our arithmetic functions saturate, let's define a helper function

```
boundInteger :: Integer -> Int
boundInteger x
  | x < toInteger intMin = intMin  -- too small, replace with intMin
  | x > toInteger intMax = intMax  -- too big, replace with intMax
  | otherwise = fromInteger x      -- just right: pass it through
```

This function takes an unboundedly-sized `Integer` and binds it to the range of a regular `Int`, using our `intMin` and `intMax` constants.

This is not very efficient: it would probably be faster to anticipate when we will saturate ahead of time, but it makes the code simple for our purposes.

Implementing Num

Now it's time to make `SatInt` an instance of `Num`.

```
instance Num SatInt where
  (SatInt x) + (SatInt y) =
    SatInt $ boundInteger $ toInteger x + toInteger y
  (SatInt x) * (SatInt y) =
    SatInt $ boundInteger $ toInteger x * toInteger y

  abs (SatInt x) =
    SatInt $ boundInteger $ abs $ toInteger x
  signum (SatInt x) = SatInt $ signum x

  negate (SatInt x) = SatInt $ boundInteger $ negate $ toInteger x

  fromInteger = SatInt . boundInteger
```

Implementing Num (2)

The most interesting lines are for `+` and `*`. We use pattern matching to pull the `Int` out of the `SatInt`. Then we convert the `x` and `y` values into unbounded `Integers`. We perform the operation.

After the operation, if the result is out of bounds, we replace it with either the max or min value as appropriate.

For `signum`, we just project the `signum` function inside the `SatInt` structure.

Implementing `Num` (3)

You might think we could do that for `abs`, too, but we can't. Suppose we're working with 8 bit integers. The range is `[-128, 127]`. If we take `abs -128`, we would expect `128`, but that's too big! We need to bound it, therefore.

Same deal with `negate`. We want `negate intMin` to give `intMax`, not wrap around to `intMin` again.

Lastly, we use point-free notation for `fromInteger`. We apply the `SatInt` constructor of the result of calling `fromInteger` on the input value.

Showing it off

```
main :: IO ()
main = do
    let x = SatInt 7
    let y = SatInt 9
    let z = SatInt 99

    print $ x + y -- Prints SatInt 16
    print $ x - y -- SatInt (-2)
    print $ x * y -- SatInt 63
    print $ abs $ x * negate y -- SatInt 63
    print (-1000000000000000000000000 :: SatInt) -- SatInt (-9223372036854775807)
    print (1000000000000000000000000 :: SatInt) -- SatInt 9223372036854775807
    print $ abs (-1000000000000000000000000 :: SatInt) -- SatInt 9223372036854775807
    print $ z*z*z*z*z*z*z*z*z*z*z*z*z*z*z*z*z*z*z*z*z -- SatInt 9223372036854775807
```

Notice the saturation being demonstrated on all the last values. Haskell has a weird length of `Int` on my platform, but it's clearly limited.

Practice

Create a custom integer type, `Int4` that ranges from -8 to +7.

Implement `Bounded` for it.

Then, implement `Num` for it, too.

Don't make it saturate, it should wrap. You can use the modulus to make it do that.

Demonstrate that your integer type works by doing some arithmetic on it.

Questions?

Other typeclasses

Now that you understand `Num`, you have a good grounding for how typeclasses work and why they're useful.

We can add any new numeric type and have it work with the operators we're used to.

We can even support higher operators like `^` if we also want to implement `Integral` (which requires `Real` and `Enum`).

Other typeclasses

Next module, we'll look at some extremely important typeclasses with have weird names: `Functor`, `Applicative`, and `Monoid`.

This will set us up to discuss one of the most important typeclasses of all: `Monad` !

One last thing: dynamic dispatch on which arg?

One random thing I wanted to point out but couldn't find a good place to do it.

In Java (and most OO languages), when we write `instance.method(1, 2, 3)`, if the type of instance is abstract (i.e., an interface or abstract class), then we're doing dynamic dispatch.

That is, assume `method` is the 3rd method in the interface. We'll look up the 3rd value in `instance`'s VTable and call it with `1, 2, 3`. This happens at run time.

But that means that only the thing before the `.` can be used in the look up.

What if?

What if we want to have the dynamic dispatch happen to another value. For example, a non-object-oriented function like this product between a scalar and a vector:

```
scale 5 (Vec3 2 3 4)
```

I might want to dispatch this function depending on whether I'm scaling a `Vec3`, a `Vec4`, a matrix, etc.

In most OO languages, we can't do this: `5.scale(Vec3(2, 3, 4))` would dispatch based on the 5. That would mean that all integers would have the same `scale` function. We couldn't have it change at runtime depending on what kind of thing we're scaling except manually (i.e., an if-statement or some kind of table of function pointers).

The value before the `.` is *special*. It is the only one allowed to cause dispatch to occur.

In Haskell, we can

```
class Scalable a where  
  scale :: Num b => b -> a -> a
```

Notice the difference, the `Scalable` thing (`a`) is the *second* argument to `scale`. The first argument is just a `Num`.

This means that we're doing dispatch-based on the second argument rather than the first. We can scale a different way for `<1, 2, 3>` and `<<1, 2,>, <3, 4>>` (pretend that's a 2x2 matrix)

Return type dispatch

We can even dispatch based on the return type.

That's how `minBound` and `maxBound` work.

In Java, you cannot overload based on return type.

This ends up being very useful in Haskell when dealing with monads, which use this feature along with `do` notation to let you extend the language in cool ways.

What about multiple dispatch

What if we want to dispatch based on *more than one* argument?

We've been assuming that we only ever want to use one argument to dispatch based on.

But what if we want different combinations of types to go to different methods?

When on earth could we ever need that?

Multiple dispatch example:

Collision detection is a good example.

There are custom math formula for detecting a rectangle-rectangle, rectangle-circle, circle-triangle, etc. collision.

So we need to consider combinations.

Haskell lets us do this with multi-parameter typeclasses:

```
class Collider a b where  
    collides :: a -> b -> Bool
```

Here, we can define a `Collider` instance for every pair we support. Like:

```
instance Collider Circle Square where  
    collides circle square = ...
```

Why have I never heard of this?

If a language requires dynamic-dispatch, multiple dispatch is pretty rare.

The [Dylan](#) language supports it...

...as does [Julia](#).

However, C++ doesn't have it. It has ad-hoc polymorphism, so you can define it on a case-by-case basis, but you have to have your definitions up-front. You can't let someone add new collisions later for example.

Java doesn't have it either. Neither does C#.

Blub strikes again

Now that you've seen it, it probably seems like the easiest way to define collisions.

But if you had never heard of multiple dispatch, it might have never even occurred to you to ask for that feature.

This is the curse of Blub, and why it's so useful to learn new languages.

Dispatch in Haskell is static

Can Haskell be blub? Yes. Here's an example.

In Haskell, dispatch is always static. We never dynamically dispatch based on the type. The compiler can always figure out which function to call at compilation, so there are no lookup tables.

However, that means we can do a particular thing in Java and not as easily in Haskell:

```
interface GuiWidget {  
    void onClick();  
}  
  
class GuiInterface {  
    static GuiWidget[] widgets = new GuiWidget[100];  
}
```

Dispatch in Haskell is static

In Haskell, the equivalent code would be this:

```
class GuiWidget a where
    onClick :: a -> IO () -- IO () is for "actions", we'll cover it later

widgets :: Widget a => [a] -- this seems to work
widgets = [Button "load", Button "quit"] -- but this fails
```

You get an error stating that `a` is a *rigid type variable*. There are two ways to explain why this fails...

Dispatch in Haskell is static (2)

1. Because the type `widget :: Widget a => [a]` really means `widget :: forall a. Widget a => [a]`. That is, it's a nonsense type that claims to be a list of every kind of `Widget`, not a specific kind. It is impossible to fulfill, because there is no bound on the kinds of `Widget` (we can define more at any time, even in external modules).
2. Because it requires dynamic dispatch. If each element of the list can be a different type, we have to, at run time, decide what to do. Therefore we need a lookup table.

Dispatch in Haskell is static (3)

So is Java just better than Haskell? At least for GUIs?

No, there are easy ways to fix this:

1. There's a language extension called `existentialQuantification` that lets you define the type of the list as "a list of *some* widget" instead of "a list of *any* widget".
2. There's a better way to design this software.

Let's focus on number 2: [can anyone think of a way to redesign our system to not rely on fancy hidden dispatch mechanisms?]

Different designs

If the main difference between widgets is what they do when you click them, why not just bake that directly into the data type as a function?

```
data Widget = Widget (Widget -> IO ())
```

Now, a `Widget` is anything that does `IO` when you click it. We can easily have a list of `Widget`s now.

We could even do this instead:

```
data Widget = Button | Scrollbar | Window | ...
```

We can use sum-types to list every kind of widget.

Different designs (2)

The second design is not very OO, but it avoids requiring a dispatch mechanism. The first design is almost exactly equivalent to the OO, but avoids the extra level of indirection.

Either way, we get a nice, simple design that doesn't require the language to secretly build lookup-tables for us.

Questions?

Class's being overly overloaded

I promised earlier that I had an example of how the design of classes needing to enable 3 different language features could cause them to not fulfill each one as well as a purpose-built solution.

Here is an example

Class layout

Classes are used for encapsulation. They act as boundaries for modifying data.

However, in most OO languages, all the data in an instance is co-located.

That is, in this Java class, when we instantiate it, `x`, `y`, and `z` will be next to each other in memory:

```
class Foo {  
    int x;  
    int y;  
    int z;  
}
```

What's wrong with that?

Nothing is wrong with that. It's what we want most of the time.

However, sometimes some subsets of fields are never used at the same time.

Consider this example:

```
class GameObject {  
    private:  
        Position pos;  
        Sprite sprite;  
        SoundEffect collisionSound;  
    public: ...  
};
```

The graphics drawing routine is probably completely unrelated to the sound processing, and yet those two pieces of data are right next to each other.

What's wrong with that? (2)

The problem with that is that it's cache-inefficient.

Having sound data in our cache when we're doing sprite drawing just reduces the amount of data that can fit in Cache.

We're probably doing a for-loop over all the sprites to draw them, then maybe a for-loop over all the sounds later to play them if a collision happened.

Therefore, we should probably store them in two separate places:

```
SoundEffect* effects;  
Sprite* sprites;
```

If each game object has a unique id, we can maybe treat it as an index into these arrays.

What's wrong with that? (3)

The issue is: this is a radical re-organization of data. We aren't using the class anymore.

We could have a class with pointers to its data, but that defeats the purpose. We really only need the class to be an id.

At this point, most game programmers switch to using an entity component system, which is based on this principle.

Fundamentally...

This is a distinction between an array of structures (or classes) and a structure of arrays.

We call this AOS vs. SOA. Sometimes one is more performant or convenient than the other.

Classes are built around an AOS assumption, which is a good assumption most of the time, but not always.

On the other hand, what if you were never using classes to begin with? Your module may have used handles for game objects with totally independent data structures.

If so, congratulations, you probably didn't break any external code when you refactored.

The point

The reason I'm mentioning this is that when we use the same feature for data-hiding as we do for controlled mutation, we may run into problems where it's not great at either.

Here, because in Haskell we don't need a feature for controlled mutation, we can just use our data-hiding tool, which is a module.

And modules are very simple. We end up with easier software design.

Questions?